

Robust Software Engineering (RSE) for Embedded Control Systems Pattern

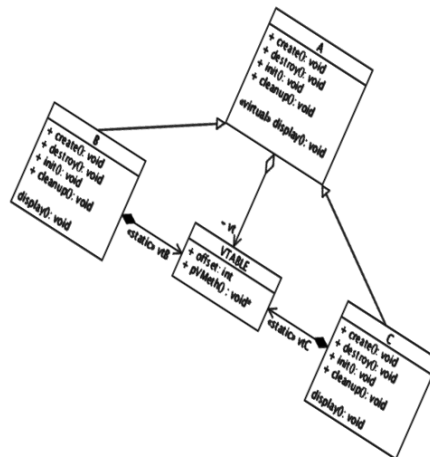
Medard Rieder

Thomas Sterren

©Haute école d'ingénierie – Intronique

Table des matières

Embedded Pattern.....	1
Modélisation de systèmes embaqués.....	1
Le modèle en oignon.....	1
Diagrammes de classes en UML.....	2
Diagrammes USE CASE en UML.....	3
Diagrammes de séquence en UML.....	3
Diagrammes d'états transitions en UML.....	4
Diagrammes de timing en UML.....	4
Ce que sont les pattern de conception	5
Le pattern "Object Oriented C"	6
Classe, attributs et méthodes.....	6
Constructeur et destructeur.....	7
Création et destruction d'objets	8
Appel des méthodes.....	9
Relations parmi classes	11
Héritage.....	15
Redéfinition des méthodes	17
Polymorphisme.....	18
Des éléments non traduits	23
Les pattern pour l'implémentation des machines d'états transitions.....	24
Machines d'états transitions	24
Le pattern "Double Switch"	24
Le pattern "Transition Method Pointer"	27
Des pattern de découplage	32
Subject – Observer orienté objets	33
Subject - Observer non orienté objet.....	36
Exercise pratique	37
Le pattern callback	38
Une variante orientée-objets du pattern callback	38
Des pattern pour l'instanciation d'objets systèmes.....	43
Le pattern Singleton.....	43
Le pattern Singleton orienté-objets	43
Le pattern Singleton non orienté-objets	44
Pattern pour la création et l'initialisation de systèmes.....	45
Le pattern Factory	45



Embedded Pattern

Dans ce chapitre sera donné une introduction aux pattern embarqués. De plus un petit moment est passé pour rappeler la technique de modélisation de systèmes embarqués.

Modélisation de systèmes embarqués

Le modèle en oignon

Lors de la modélisation d'un système embarqué, il est recommandé de respecter le modèle dit "en oignon". Le diagramme suivant montre ce modèle :

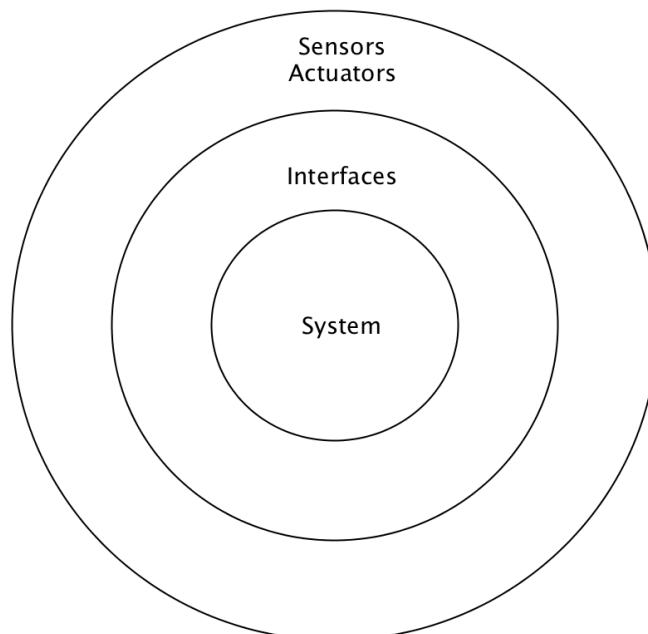


Figure 1 : Modèle en oignon

Avec ce modèle, le système est subdivisé en une logique centrale (Système), une couche de communication (Interfaces) et une couche périphérique (Capteurs, actuateurs). Une telle subdivision est avantageuse car elle inclut d'une part le point de vue de l'utilisateur, et d'autre part des éléments importants tels que les bus de communication et les interfaces de communication. Ainsi ce modèle peut supporter et accompagner le processus de développement durant les phases orientées vers utilisateur comme durant les phases orientées technique. L'exemple d'un ascenseur est présenté sur la figure suivante :

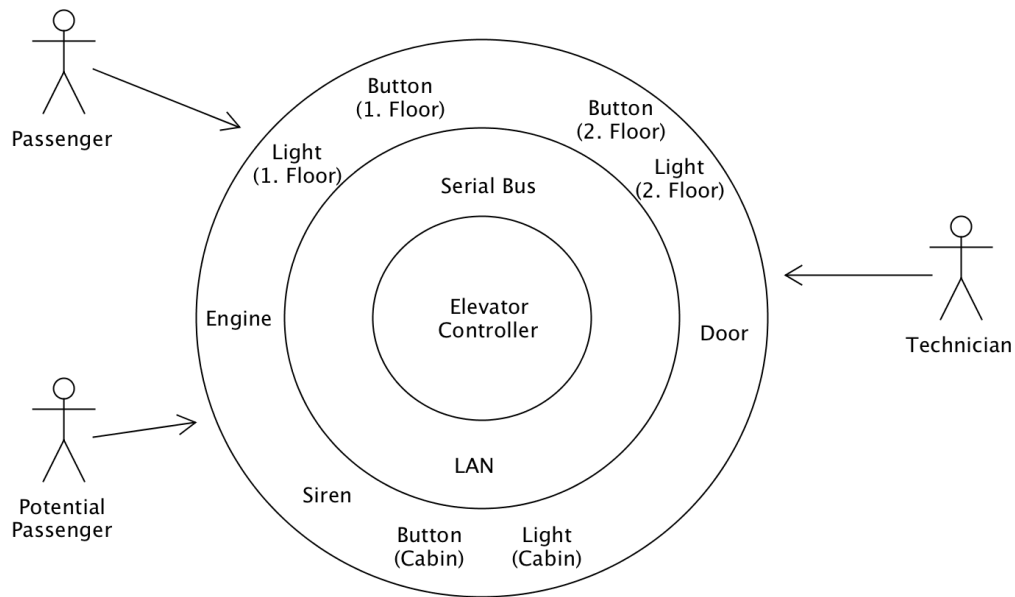


Figure 2: Modèle en oignon d'un ascenseur

Diagrammes de classes en UML

Un autre avantage de ce système est sa transposition simple en UML dans un diagramme de paquets. La figure suivante représente le même modèle représenté en UML.

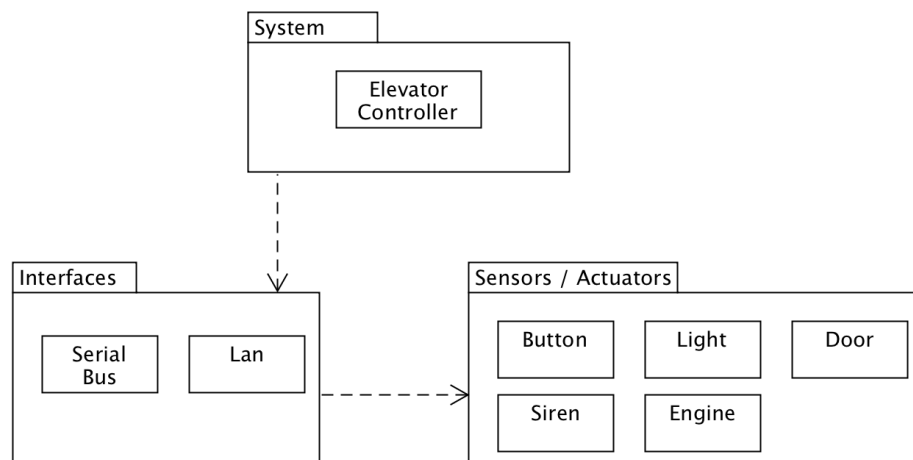


Figure 3 : Diagramme UML de classes d'un ascenseur

Diagrammes USE CASE en UML

La figure suivante montre un diagramme UML de type USE CASE, un autre élément très important pour la modélisation des systèmes embarqués :

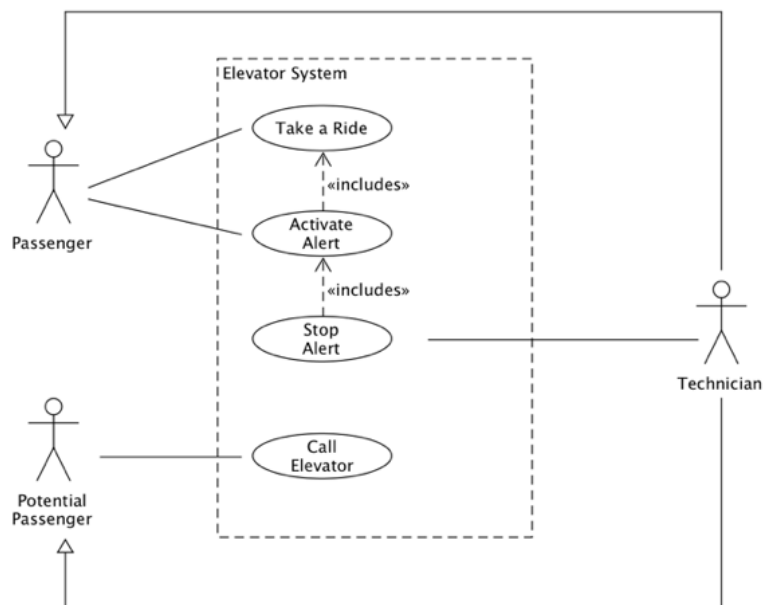


Figure 4 : Diagramme UML USE CASE d'un ascenseur

Diagrammes de séquence en UML

La figure suivante montre un diagramme UML de séquence qui est le type de diagramme les plus important pour la modélisation de systèmes embarqués.

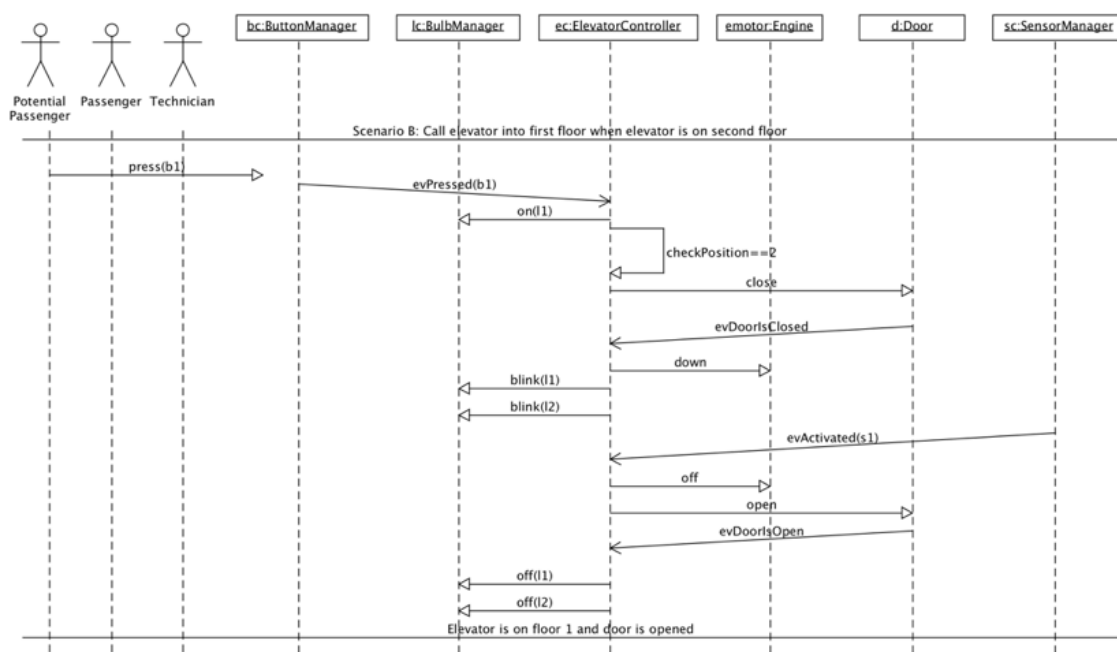


Figure 5 : Diagramme UML de séquence d'un ascenseur

Diagrammes d'états transitions en UML

Le diagramme en UML le plus important pour le développement des systèmes embarqués est certainement le diagramme d'états transitions. Ce diagramme montre, comment un objet peut réagir à des différents événements, dans quel état il transite et quelles actions il exécute pendant ce temps. La figure suivante montre le diagramme d'états transitions d'un simple system qui peut être enclenché et déclenché.

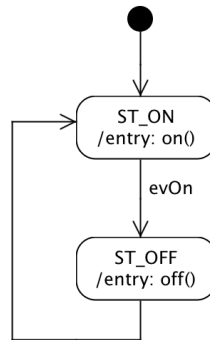


Figure 6 : Diagramme UML d'états transitions

Diagrammes de timing en UML

Pour le développement des systèmes temps réel, les diagrammes de timing sont très importants. Ces derniers montrent les aspects temporels d'un système temps réel et sont alors très importants pour le développeur. La figure ci-dessous montre un tel diagramme de timing en UML :

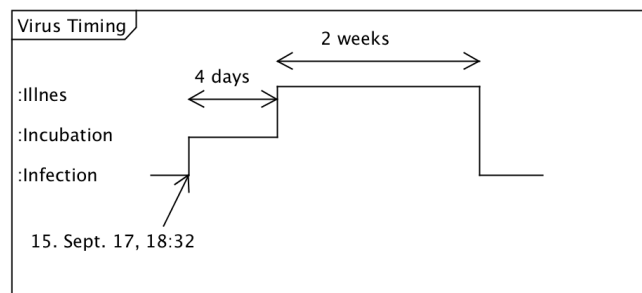


Figure 7 : Diagramme UML de timing

Ils y existent encore toute une série d'autres type de diagrammes UML qui seront expliqués en détails dans des chapitres suivants.

Le développement des systèmes embarqués et temps réel n'est pas seulement réalisé en diagrammes UML mais il y a aussi un grand nombre de pattern qui sont utilisés. Ces derniers seront traités un peu plus en détail dans les sections suivantes.

Ce que sont les pattern de conception

Les pattern de conception sont des solutions efficaces pour certains problèmes connus. Il existe une multitude de pattern de conception et les aborder tous dépasseraient de loin le cadre de ce cours. Nous ne donnerons ici qu'un petit aperçu des pattern existants. En principe, chaque pattern de conception peut être décrit par les informations suivantes :

- Nom
- Description du problème traité
- Description des solutions
- Conséquences

Les pattern peuvent être classés comme suit :

Embedded Pattern Overview	Creational	Structural	Behavioral	Decoupling	Implementation
Object Oriented C					X
Double Switch			X		X
Transition Method Pointer			X		X
Subject - Observer		X	X	X	
Callback Method		X		X	
Sigleton	X				
Factory	X	X			

Tableau 1 : Classement des pattern

La liste ci-dessus n'est pas complète. Elle montre un extrait avec les pattern les plus importants seulement.

Ensuite, un nombre de pattern seront montrés plus en détail. Ces pattern sont partiellement adaptés aux besoins du développement des systèmes embarqués.

Le pattern "Object Oriented C"

La traduction d'un modèle UML en un langage orienté objet ne représente pas un grand problème. Ceci n'est par contre pas le cas, si le langage cible n'est pas orienté objet, comme par exemple pour C. Pour des systèmes embarqués, le C est pour différentes raisons toujours un langage très important. C'est pourquoi cette section va présenter comment UML est traduit en code C. La traduction des éléments suivants sera présentée :

- Classe avec attributs et méthodes
- Constructeur et destructeur d'une classe
- Création d'objets
- Appel de méthode
- Relations parmi des classes
- Héritage
- Polymorphisme
- Machines d'états transition
- Éléments qui ne sont pas traduits

Classe, attributs et méthodes

Une classe est traduite en créant une structure C contenant tous les attributs de celle-ci. La structure porte le même nom que la classe elle-même. Pour chaque méthode, une fonction C est créée. Elle a le nom `Class_method(Class* const me, Type Param, ... , Type Param)`. Techniquement, deux fichiers sont créés :

- un fichier `*.h` portant le nom `class.h`
- et un fichier `*.c` portant le nom `class.c`

Le fichier `*.h` contient la déclaration de la structure et les prototypes des méthodes. La règle suivante est à respecter : Les déclarations des méthodes public se trouvent dans le fichier `*.h` et les méthodes privées existent seulement dans le fichier `*.c`. La figure suivante montre la classe A en UML et en C :


```
#ifndef CLASS_A
#define CLASS_A

/* this is the forward declaration
 * of the class A attributes
 */

typedef struct A
{
    int m;
    int n;
} A;

/* public methods
 */

void A_setM(A* const me, int p);
void A_display(A* const me);

#endif

#include "a.h"
#include <stdio.h>
#include <stdlib.h>

/* this are the declarations
 * of the private methods
 */
void A_calculate(A* const me);

/* here come the implementations
 */

void A_setM(A* const me, int p)
{
    me->m = p;
    A_calculate(me);
}

void A_display(A* me)
{
    printf("v1=%d, v2=%d\n", me->m, me->n);
}

void A_calculate(A* const me)
{
    me->n = 2 * me->m;
}
```

Figure 8 : La classe A en Code C

Le graphique suivant montre le diagramme UML de la classe A:

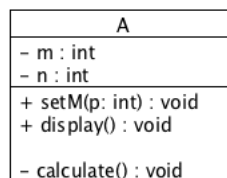


Figure 9 : Le diagramme UML de la classe A

Constructeur et destructeur

Le constructeur et le destructeur sont réalisés au moyen des méthodes `X_Init()` und `X_Cleanup()` avec `X` étant le nom de la classe. Un seul constructeur est généré car le langage C ne connaît pas la notion de surcharge d'une méthode. Ce constructeur peut être un avec ou sans paramètres. Le type de retour de ces 2 fonctions est `void`. Le système génère les noms des fonctions de la même façon qu'avec les autres fonctions, à savoir :

- `X_Init(X* const me, Type Param, ... , Type Param)`
- `X_Cleanup(X* const me)`

Les images ci-dessous montrent le code et le diagramme UML d'une classe :

```
#ifndef CLASS_A
#define CLASS_A

/* this is the forward declaration
 * of the class A attributes
 */

typedef struct A
{
    int m;
    int n;
} A;

/* public methods
 */
void A_init(A * const me);
void A_cleanup(A * const me);

void A_setM(A* const me, int p);
void A_display(A* const me);

#endif

#include "a.h"
#include <stdio.h>
#include <stdlib.h>

/* this are the declarations
 * of the private methods
 */
void A_calculate(A* const me);

/* here come the implementations
 */
void A_init(A* const me)
{
    me->m = 0;
    me->n = 0;
}

void A_cleanup(A* const me)
{
    //nothing to do, no dynamic memory
}
```

Figure 10 : Constructeur et destructeur de la classe A en code C

A
- m : int - n : int
+ init(); + cleanup(); + setM(p: int) : void + display() : void - calculate() : void

Figure 11 : Les méthodes init() et cleanup() de la classe A

Création et destruction d'objets

Dans une classe, deux fonctions sont générées qui s'appellent `X_create()` et `X_destroy()`. La fonction `X_create()` génère une structure qui contient les attributs de la classe alloués en mémoire dynamique et exécute le constructeur (`X_init()`). On obtient comme valeur de retour un pointeur sur cette structure. La destruction d'un objet est effectuée au moyen de la fonction `X_destroy()` qui appelle la fonction du destructeur (`X_cleanup()`), ce qui a pour effet de libérer l'espace mémoire correspondant à la structure qui était créée. Si les objets ne sont pas créés de manière dynamique, il faut appeler les fonctions `X_init()` et `X_cleanup()` explicitement.

Il est alors ainsi que la fonction `X_create()` ensemble avec la fonction `X_init()` -forme un espèce de constructeur. Au cas où la fonction `X_init()` -demande un ou des paramètres, il sera nécessaire d'équiper la fonction `X_create()` avec les mêmes paramètres. Le même peut être dit pour la fonction `X_destroy()` -laquelle forme ensemble avec la fonction `X_cleanup()` un espèce de destructeur.

Les figures suivantes montrent de quelle façon fonctionnent les fonctions `A_create()` et `A_destroy()` relatives à la création et à la destruction d'un objet et un diagramme UML.

```
#ifndef CLASS_A
#define CLASS_A

/* this is the forward declaration
 * of the class A attributes
 */

typedef struct A
{
    int m;
    int n;
} A;

/* public methods
 */
A* A_create();
void A_destroy(A* const me);
void A_init(A * const me);
void A_cleanup(A * const me);

void A_setM(A* const me, int p);
void A_display(A* const me);

#endif

#include "a.h"
#include <stdio.h>
#include <stdlib.h>

/* this are the declarations
 * of the private methods
 */
void A_calculate(A* const me);

/* here come the implementations
 */

A* A_create()
{
    A* newObj = (A*) malloc(sizeof(A));
    if (newObj != NULL)
    {
        A_init(newObj);
    }
    return newObj;
}

void A_destroy(A* const me)
{
    if (me != NULL)
    {
        A_cleanup(me);
    }
    free(me);
}
```

Figure 12 : Le code des méthodes `create()` et `destroy()` de la classe A en code C

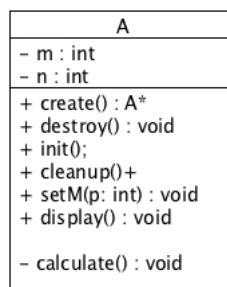


Figure 13 : Le diagramme UML de la classe A

Appel des méthodes

L'appel d'une méthode d'une classe est représenté à la figure suivante :

- `retval = Class_method(objectPointer, param, ... , param);`
- `Class_method(objectPointer, param, ... , param);`

Les deux graphiques suivants montrent la création, l'utilisation et la destruction d'un objet de la classe A et le diagramme UML correspondant.

```
#include <stdio.h>
#include "a.h"

int main(void)
{
    A* objA1 = A_create(); //this implicitly calls A_init()
    A* objA2 = A_create();
    A objA3;               //this does NOT call A_init()
    A_init(&objA3);        //so you have to call it yourself

    A_setM(objA1, 10);     //call setM Method with objA1
    A_setM(objA2, 3);      //call setM Method with objA2
    A_setM(&objA3, 20);    //call setM Method with objA3

    A_display(objA1);      //call display Method with objA1
    A_display(objA2);      //call display Method with objA2
    A_display(&objA3);     //call display Method with objA3

    A_destroy(objA2);      //this implicitly calls A_cleanup();
    A_destroy(objA1);

    //A_destroy(&objA3);    //do not try to do this!
    A_cleanup(&objA3);     //but this one is fine
    return 0;
}
```

Figure 14 : Un programme de test pour la classe A

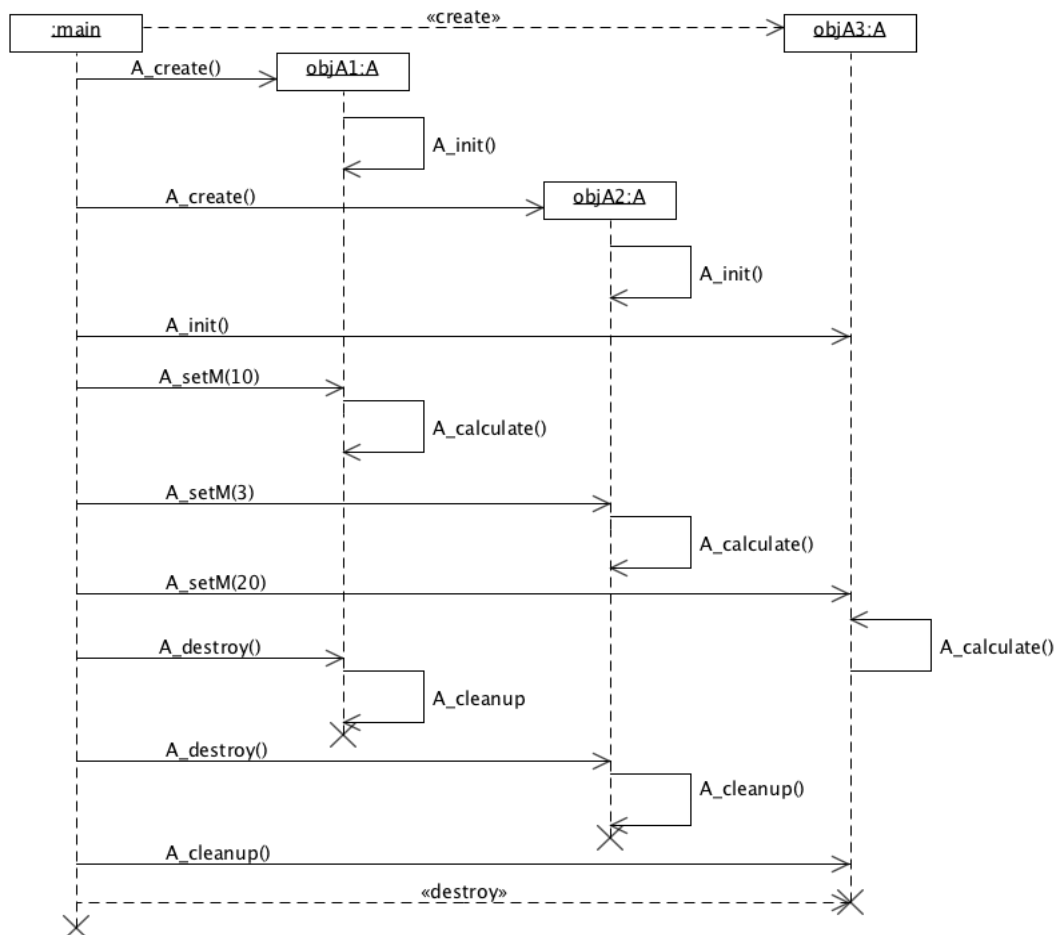


Figure 15 : Le diagramme de séquence du code montré par la figure précédente

Relations parmi classes

Les relations entre classes sont traduites de la façon suivante :

- **Dependency:** est traduite avec un `#include`
- **Association:** est en général traduite au moyen d'un pointeur
- **Aggregation:** est toujours traduit au moyen d'un pointeur. Ce dernier est automatiquement initialisé par `NULL` dans le constructeur (`X_init()`). Les méthodes `X_cleanup()` qui est appelée par le destructeur sert à réinitialiser ces pointeurs.
- **Composition:** est traduite au moyen d'une instantiation de la classe dans les attributs (membre dans le `struct`). Dans le constructeur, la fonction `X_init()` est appelée. Celle-ci appelle le constructeur de la classe instanciée. Lors de l'appel de la fonction `X_cleanup()`, c'est le destructeur de la classe instanciée qui est appelé.

Les diagrammes suivants montrent les éléments correspondants :

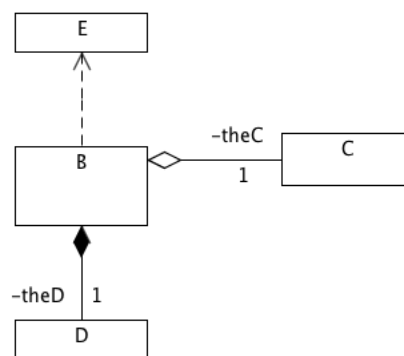


Figure 16 : Diagramme UML avec des classes ayant des relations

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

typedef struct _E
{
} E;

void E_init(E * const me);
void E_cleanup(E * const me);
E* E_create();
void E_destroy(E* const me);

void E_display(E* const me);

#include "E.h"
#include <stdio.h>
#include <stdlib.h>

E* E_create()
{
    E* newObj = (E*) malloc(sizeof(E));
    if (newObj != NULL)
    {
        E_init(newObj);
    }
    return newObj;
}

void E_destroy(E* const me)
{
    if (me != NULL)
    {
        E_cleanup(me);
    }
    free(me);
}

void E_init(E* const me)
{
}

void E_cleanup(E* const me)
{
}

void E_display(E* const me)
{
    printf("this is an E object\n");
}
```

Figure 17 : Le code C de la classe E

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

typedef struct _D
{
} D;

void D_init(D * const me);
void D_cleanup(D * const me);
D* D_create();
void D_destroy(D* const me);

#include "D.h"
#include <stdlib.h>

D* D_create()
{
    D* newObj = (D*) malloc(sizeof(D));
    if (newObj != NULL)
    {
        D_init(newObj);
    }
    return newObj;
}

void D_destroy(D* const me)
{
    if (me != NULL)
    {
        D_cleanup(me);
    }
    free(me);
}

void D_init(D* const me)
{
}

void D_cleanup(D* const me)
{
}
```

Figure 18 : Le code C de la classe D

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

typedef struct _C
{
} C;

void C_init(C * const me);
void C_cleanup(C * const me);
C* C_create();
void C_destroy(C* const me);

void C_display(C* const me);

#include "C.h"
#include <stdlib.h>
#include <stdio.h>

C* C_create()
{
    C* newObj = (C*) malloc(sizeof(C));
    if (newObj != NULL)
    {
        C_init(newObj);
    }
    return newObj;
}

void C_destroy(C* const me)
{
    if (me != NULL)
    {
        C_cleanup(me);
    }
    free(me);
}

void C_init(C* const me)
{
}

void C_cleanup(C* const me)
{
}

void C_display(C* const me)
{
    printf("this is a C object\n");
}
```

Figure 19 : Le code C de la classe C

```
#pragma once

#ifndef NULL
#define NULL 0
#endif
#include "c.h"
#include "d.h"
#include "e.h"

typedef struct _B
{
    C* theC;
    D theD;
} B;

void B_init(B * const me);
void B_cleanup(B * const me);
B* B_create();
void B_destroy(B* const me);

void B_setC(B* const me, C* anC);
C* B_getC(B* const me);
void B_useSomeE(B* const me, E* const anE);

#include "B.h"
#include <stdlib.h>

B* B_create()
{
    B* newObj = (B*) malloc(sizeof(B));
    if (newObj != NULL)
    {
        B_init(newObj);
    }
    return newObj;
}

void B_destroy(B* const me)
{
    if (me != NULL)
    {
        B_cleanup(me);
    }
    free(me);
}

void B_init(B* const me)
{
    me->theC = NULL;
    D_init(&me->theD);
}

void B_cleanup(B* const me)
{
    me->theC = NULL;
    D_cleanup(&me->theD);
}

void B_setC(B* const me, C* anC)
{
    me->theC = anC;
}

C* B_getC(B* const me)
{
    return me->theC;
}

void B_useSomeE(B* const me, E* const anE)
{
    E_display(anE);
}
```

Figure 20 : Le code C de la classe B

```
#include "b.h"
#include "c.h"
#include "e.h"

int main(void)
{
    B* objB = B_create();           //create a B object, this creates also a D object
    C* objC = C_create();           //create a C object
    E* objE = E_create();           //create a D object
    C* tempC;                       //a temporary C pointer
    B_setC(objB, objC);              //reference theC to a true object
    tempC = B_getC(objB);            //get theC out of B
    C_display(tempC);                //call some method of the C class using this pointer
    B_useSomeE(objB, objE);          //the B class uses some E object
    B_destroy(objB);                 //destroy the B object, this also will destroy the D
    C_destroy(objC);                 //destroy the C object
    E_destroy(objE);                 //destroy the E object
    return 0;
}
```

Figure 21 : Un programme de test pour les classes B, C, D et E

Héritage

L'héritage est traité de façon quasi identique à la composition, à savoir que la classe spécialisée contient un objet de la classe de base de laquelle il hérite. Par conséquent il est responsable d'invoquer le constructeur et destructeur avec les bons paramètres et donc d'initialiser correctement l'objet de la classe de base ainsi que de le détruire. Les schémas ci-dessous montrent un exemple d'héritage et le diagramme de classe correspondant.

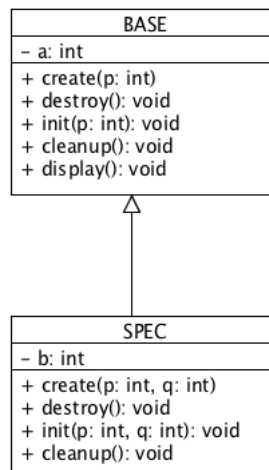


Figure 22 : Le diagramme UML des classes BASE et SPEC

```

#pragma once

#ifndef NULL
#define NULL 0
#endif

typedef struct _BASE
{
    int a;
} BASE;

void BASE_init(BASE * const me, int p);
void BASE_cleanup(BASE * const me);
BASE* BASE_create(int p);
void BASE_destroy(BASE* const me);
void BASE_display(BASE* const me);

#include "base.h"
#include <stdio.h>
#include <stdlib.h>

BASE* BASE_create(int p)
{
    BASE* newObj = (BASE*) malloc(sizeof(BASE));
    if (newObj != NULL)
    {
        BASE_init(newObj, p);
    }
    return newObj;
}

void BASE_destroy(BASE* const me)
{
    if (me != NULL)
    {
        BASE_cleanup(me);
    }
    free(me);
}

void BASE_init(BASE* const me, int p)
{
    me->a = p;
}

void BASE_cleanup(BASE* const me)
{
}

void BASE_display(BASE* const me)
{
    printf("BASE::a = %d\n", me->a);
}
    
```

Figure 23 : Le code de la classe BASE

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

#include "base.h"

typedef struct _SPEC
{
    BASE super;
    int b;
} SPEC;

void SPEC_init(SPEC * const me, int p, int q);
void SPEC_cleanup(SPEC * const me);
SPEC* SPEC_create(int p, int q);
void SPEC_destroy(SPEC* const me);

#include "spec.h"
#include <stdlib.h>

SPEC* SPEC_create(int p, int q)
{
    SPEC* newObj = (SPEC*) malloc(sizeof(SPEC));
    if (newObj != NULL)
    {
        SPEC_init(newObj, p, q);
    }
    return newObj;
}

void SPEC_destroy(SPEC* const me)
{
    if (me != NULL)
    {
        SPEC_cleanup(me);
    }
    free(me);
}

void SPEC_init(SPEC* const me, int p, int q)
{
    //call the base class constructor
    BASE_init(&me->super, p);
    me->b = q;
}

void SPEC_cleanup(SPEC* const me)
{
    //call the base class destructor
    BASE_cleanup(&me->super);
}
```

Figure 24 : Le code de la classe SPEC qui hérite de la classe BASE

```
#include "base.h"
#include "spec.h"

int main(void)
{
    BASE* baseObj1 = BASE_create(1); //create dynamic object of base class
    BASE baseObj2; //create static object of base class
    BASE_init(&baseObj2,2); //do not forget to initialize it

    SPEC* specObj1 = SPEC_create(3,4); //create dynamic object of specialized class
    SPEC specObj2; //create static object of specialized class
    SPEC_init(&specObj2,5,6); //do not forget to initialize it

    BASE_display(baseObj1); //work with the class
    BASE_display(&baseObj2);

    BASE_display(&specObj1->super); //call a method of the base class
    BASE_display(&specObj2.super);

    BASE_destroy(baseObj1); //remove dynamic base class object
    BASE_cleanup(&baseObj2); //cleanup static base class object
}
```

Figure 25 : Le code C d'une application avec les classes BASE et SPEC

Redéfinition des méthodes

Il est naturellement possible de redéfinir les attributs et les méthodes dans la classe spécialisée. Cela se fait en déclarant/définissant un attribut/méthode de la classe de base dans la classe spécialisée. Le programmeur doit ensuite utiliser les appels correspondants et s'assurer qu'il utilise les bons attributs/méthodes. Une méthode spécialisée peut également appeler sa version dans la classe de base. Les illustrations ci-dessous montrent un exemple en UML et en code.

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

#include "base.h"

typedef struct _SPEC
{
    BASE super;
    int b;
    int a;    //redefinition of a
} SPEC;

void SPEC_init(SPEC * const me, int p, int q, int r);
void SPEC_cleanup(SPEC * const me);
SPEC* SPEC_create(int p, int q, int r);
void SPEC_destroy(SPEC* const me);

void SPEC_display(SPEC * const me);

#include "spec.h"
#include <stdlib.h>
#include <stdio.h>

SPEC* SPEC_create(int p, int q, int r)
{
    SPEC* newObj = (SPEC*) malloc(sizeof(SPEC));
    if (newObj != NULL)
    {
        SPEC_init(newObj, p, q, r);
    }
    return newObj;
}

void SPEC_destroy(SPEC* const me)
{
    if (me != NULL)
    {
        SPEC_cleanup(me);
    }
    free(me);
}

void SPEC_init(SPEC* const me, int p, int q, int r)
{
    //call the base class constructor
    BASE_init(&me->super, p);
    me->b = q;
    me->a = r;
}

void SPEC_cleanup(SPEC* const me)
{
    //call the base class destructor
    BASE_cleanup(&me->super);
}

void SPEC_display(SPEC* const me)
{
    printf("*** printed from SPEC ***\n");
    //call the base class method
    BASE_display(&me->super);
    printf("SPEC::a = %d\n", me->a);
}
```

Figure 26 : code de la classe SPEC avec redéfinition d'une méthode

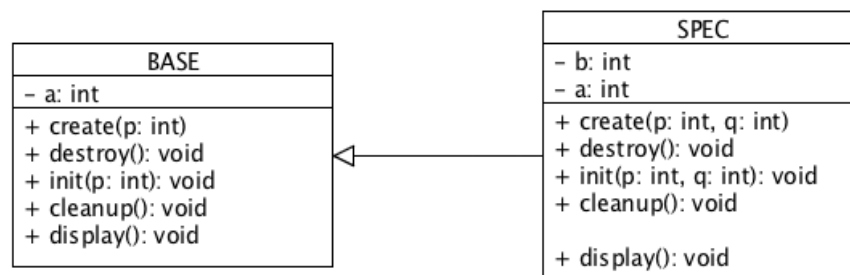


Figure 27 : Diagramme UML de la classe SPEC étendue

```
#include "spec.h"

int main(void)
{
    SPEC* specObj1 = SPEC_create(3,4,5); //create dynamic object of specialized class
    SPEC specObj2;                       //create static object of specialized class
    SPEC_init(&specObj2,6,7,8);          //do not forget to initialize it

    BASE_display(&specObj1->super);      //call a method of the base class
    BASE_display(&specObj2.super);

    SPEC_display(specObj1);              //call the redefined method of the
    SPEC_display(&specObj2);             //specialized class

    SPEC_destroy(specObj1);              //remove dynamic specialized class object
    SPEC_cleanup(&specObj2);             //cleanup static specialized class object
}
```

Figure 28 : Code C du programme de test pour la classe SPEC étendue

Polymorphisme

Il existe différents pattern pour représenter le polymorphisme en C. Un des plus simples consiste sans doute à définir des pointeurs de fonction sur des méthodes homonymes. Le pointeur peut ensuite désigner plusieurs méthodes polymorphes, ce qui permet de simuler une sorte de polymorphisme. La variante montrée ici utilise une sorte de VTABLE. Il s'agit d'une table où les pointeurs de fonction sont liés à un offset qui permet de trouver et d'utiliser automatiquement la bonne version. L'illustration suivante montre les classes A, B et C. Nous admettons que la méthode `display()` est polymorphe et qu'elle peut être invoquée dans B ou C via un pointeur de fonction dans la classe de base :

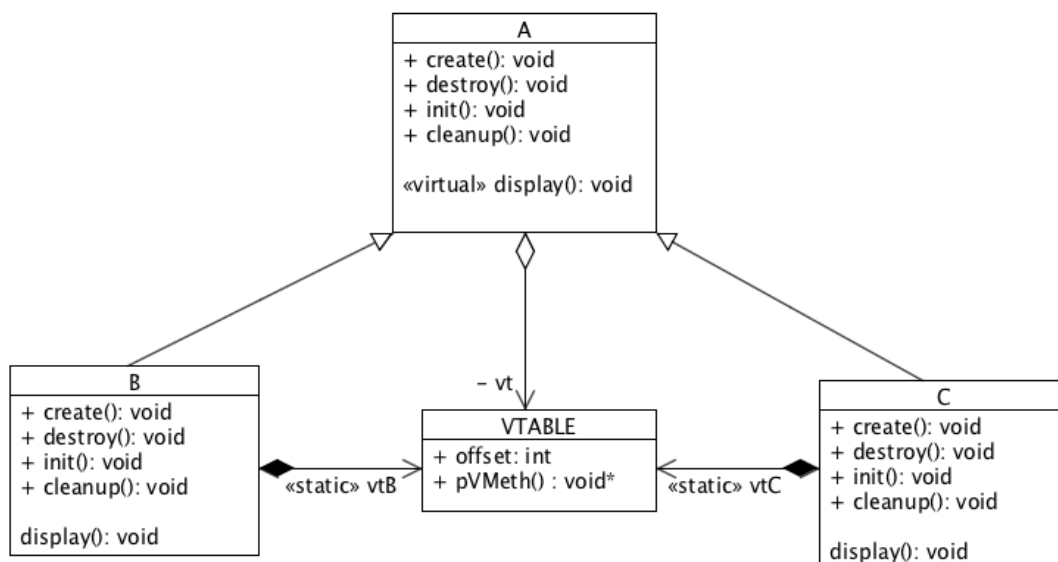


Figure 29 : Diagramme UML des classes A, B et C avec la méthode polymorphe `display()` et une sorte de VTABLE

Les illustrations suivantes montrent le code des classes A, B et C ainsi que les commentaires qui en découlent.

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

#define addrtype long

//this is the VTABLE
//it contains an offset
//and a method pointer

typedef struct _VTABLE
{
    addrtype offset;
    void (*vMeth)();
} VTABLE;

typedef struct _A
{
    //the baseclass contains
    //only a pointer to the
    //real vtables. The address
    //is passed by the constructor
    VTABLE* vt;
} A;

void A_init(A * const me, VTABLE* vt);
void A_cleanup(A * const me);
A* A_create(VTABLE* vt);
void A_destroy(A* const me);

//here comes the polymorph method
void A_display(A* me);

#include "polya.h"
#include <stdlib.h>
#include <stdio.h>

A* A_create(VTABLE* vt)
{
    A* newObj = (A*) malloc(sizeof(A));
    if (newObj != NULL)
    {
        A_init(newObj, vt);
    }
    return newObj;
}

void A_destroy(A* const me)
{
    if (me != NULL)
    {
        A_cleanup(me);
    }
    free(me);
}

void A_init(A* const me, VTABLE* vt)
{
    //the childclass calling the
    //constructor of the baseclass
    //send a pointer to it's VTABLE
    me->vt = vt;
}

void A_cleanup(A* const me)
{
    me->vt = NULL;
}

void A_display(A* me)
{
    //calculate the startaddress of the calling object
    void* objAddr = (void*) ((addrtype)me - me->vt->offset);
    //call the method and pass the objects address
    //as parameter. This is the polymorphism
    me->vt->vMeth(objAddr);
}
```

Figure 30 : Le code C de la classe A avec la méthode polymorphe display ()

La classe A est une classe toute simple. Les seuls éléments particuliers sont un pointeur sur une instance de VTABLE et méthode `A_display` qui contient un code très particulier. Cette méthode contient un pointeur sur une instance de A, dans lequel "vivent" une instance de B ou C. Grâce à l'instance de VTABLE adressée par `vt` nous pouvons calculer l'adresse de B ou C. Cette valeur est passée en paramètre à la méthode désignée par le pointeur `vMeth` se trouvant également dans l'instance de VTABLE pointée par `vt`. Cela permet d'implémenter une sorte de comportement polymorphique qui se propage à travers un nombre arbitraire de niveaux d'héritage.

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

#include "polya.h"

typedef struct _B
{
    int val;
    A super;
} B;

void B_init(B * const me, int val);
void B_cleanup(B * const me);
B* B_create(int val);
void B_destroy(B* const me);

void B_display(B* const me);

#include "polyb.h"

B* B_create(int val)
{
    B* newObj = (B*) malloc(sizeof(B));
    if (newObj != NULL)
    {
        B_init(newObj, val);
    }
    return newObj;
}

void B_destroy(B* const me)
{
    if (me != NULL)
    {
        B_cleanup(me);
    }
    free(me);
}

void B_init(B* const me, int val)
{
    //create the one and only vtable
    // common to all B-type objects.
    static VTABLE BVT;
    //where starts the base class part
    BVT.offset = offsetof(B, super);
    //the address of the B class version
    //of the display method
    BVT.vMeth = &B_display;
    //send everything to the base class
    A_init(&me->super, &BVT);
    //fianlly initialize B class attributes
    me->val = val;
}

void B_cleanup(B* const me)
{
}

void B_display(B* const me)
{
    printf("B::val = %d\n", me->val);
}
```

Figure 31 : Le code C de la classe B avec la méthode polymorphe display ()

La méthode `B_init` crée une instance statique `BVT` de type `VTABLE` et initialise un pointeur de `VTABLE` dans la classe de base `A`. Cette variable est commune à tous les objets `B` et contient les informations sur l'offset de l'instance de la classe de base `super` à l'intérieur de `B` ainsi que l'adresse de la méthode polymorphe `B_display()`. Quand on connaît l'offset de l'instance `A super`, on peut la placer où l'on veut à l'intérieur d'une structure `B`. Une possibilité sera aussi de la placer au début de la structure `B`, pour ne plus avoir besoin de l'offset. Le risque d'erreur de placement doit toutefois être pris en compte.

Le reste du code est le standard d'une classe simple implémentée en C.


```
#pragma once

#ifndef NULL
#define NULL 0
#endif

#include "polya.h"

typedef struct _C
{
    float val;
    A super;
} C;

void C_init(C * const me, float val);
void C_cleanup(C * const me);
C* C_create(float val);
void C_destroy(C* const me);

void C_display(C* const me);

#include "polyc.h"
#include <stdlib.h>
#include <stdio.h>
#include <stddef.h>

C* C_create(float val)
{
    C* newObj = (C*) malloc(sizeof(C));
    if (newObj != NULL)
    {
        C_init(newObj, val);
    }
    return newObj;
}

void C_destroy(C* const me)
{
    if (me != NULL)
    {
        C_cleanup(me);
    }
    free(me);
}

void C_init(C* const me, float val)
{
    //create the one and only vtable
    // common to all C-type objects.
    static VTABLE CVT;
    //where starts the base class part
    CVT.offset = offsetof(C, super);
    //the address of the B class version
    //of the display method
    CVT.vMeth = &C_display;
    //send everything to the base class
    A_init(&me->super, &CVT);
    //fianlly initialize B class attributes
    me->val = val;
}

void C_cleanup(C* const me)
{
}

void C_display(C* const me)
{
    printf("C::val = %f\n", me->val);
}
```

Figure 32 : Le code C de la classe C avec la méthode polymorphe `display()`

La méthode `C_init` crée une instance statique `CVT` de type `VTABLE` et initialise un pointeur de `VTABLE` dans la classe de base `A`. Cette variable est commune à tous les objets `C` et contient les informations sur l'offset de l'instance de la classe de base `super` à l'intérieur de `C` ainsi que l'adresse de la méthode polymorphe `C_display()`. Quand on connaît l'offset de l'instance `A super`, on peut la placer où l'on veut à l'intérieur d'une structure `C`. Une possibilité sera aussi de la placer au début de la structure `C`, pour ne plus avoir besoin de l'offset. Le risque d'erreur de placement doit toutefois être pris en compte.

Le reste du code est le standard d'une classe simple implémentée en C

```
#include "polya.h"
#include "polyb.h"
#include "polyc.h"

int main(void)
{
    //create objects
    B* pObj1 = B_create(33);
    C* pObj2 = C_create(3.14159);

    //this is a base class pointer
    A* pA;

    //assign address to base class pointer
    pA = &pObj1->super;
    //display once again using a B object
    A_display(pA);

    //assign address to base class pointer
    pA = &pObj2->super;
    //display once again using a C objects
    A_display(pA);

    //destroy objects
    B_destroy(pObj1);
    C_destroy(pObj2);
}
```

Figure 33 : Le code C du programme de test pour le polymorphisme

Un pointeur de type A pointe pour commencer sur un objet de type B, plus précisément sur son attribut A `super`. C'est possible car la classe B hérite de la classe A. A l'appel de la méthode `A_display` les informations contenues dans les instances `BVT`, respectivement `CVT`, sauvegardés dans la `VTABLE` sont utilisées pour trouver l'adresse de l'objet appelant ainsi que l'adresse correcte de la méthode virtuelle. C'est pour cela que nous appelons une fois la méthode `B_display` et une autre fois la méthode `C_display`. La valeur 33 est affichée, puis la valeur 3.14159.

Des éléments non traduits

Les éléments suivants ne sont généralement pas traduits en code C:

- **Visibilité :** Il n'existe pas de pattern connu pour implémenter la visibilité `public`, `private` et `protected` en C. La seule possibilité consiste à déplacer les données du `.h` dans le `.c`. Tout ce qui se trouve dans le `.h` serait considéré comme `public`, et ce qui se trouve dans le `.c` serait `private`.
- **Surcharge :** Comme C ne supporte pas la surcharge, il n'est que possible de le simuler de façon indirecte.
- **Templates :** Il n'existe pas de pattern connu pour implémenter les templates en C. Si l'on essayait, le chemin passerait par de nombreux pointeurs `void*` et encore plus de conversion de type. Cela rendra le code quelque peu "boiteux" et difficilement lisible, ce qui est bien sûr à éviter.

Exceptions : Comme les exceptions ont leur processus propre, à savoir qu'elles s'exécutent en parallèle au processus à surveiller, il serait possible de créer une structure semblable en C en traitant les erreurs dans une Thread/machine d'état séparée. Cela nécessite soit un système d'exploitation temps réel avec des Threads ou un XF pour les machines d'état. Les deux sont possibles mais nécessitent une mise en œuvre assez lourde. Le traitement des erreurs se fait généralement en contrôlant la valeur de retour des fonctions. (C'est également aussi le cas en C++)

Les pattern pour l'implémentation des machines d'états transitions

Machines d'états transitions

Une classe réactive contient une machine d'état, décrite à l'aide d'un diagramme d'état. Il existe différents pattern pour traduire une machine d'état, parmi lesquels certains ne peuvent être traduits correctement qu'en utilisant un langage orienté objet. Nous allons connaître ici deux pattern simples qui peuvent se traduire soit en langage non orienté objet soit en langage orienté objets.

Le pattern "Double Switch"

Dans ce pattern, la traduction d'une machine d'état résulte principalement en une méthode avec la signature suivante :

- `void X_processEvent(X* const me, Event * const ev)`

Cette fonction contient deux parties essentielles :

- Une structure de contrôle `switch`, qui calcule le nouvel état de la machine d'état en fonction de son état courant et l'événement `ev`.
- Une seconde structure `switch` qui exécute les actions contenues dans `action on exit` de l'ancien état et l'action `on entry` l'état suivant.

La classe doit également contenir la définition d'une énumération symbolisant les différents états avec un nom si possible parlant. La classe contient aussi un élément `currentState` du même type que l'énumération des états. Une méthode `getCurrentState()` permet la lecture de cet attribut.

Le schéma suivant montre une machine d'état en UML et le code correspondant en C. Il est bien clair que ce schéma UML pourrait aussi être traduit en C++.

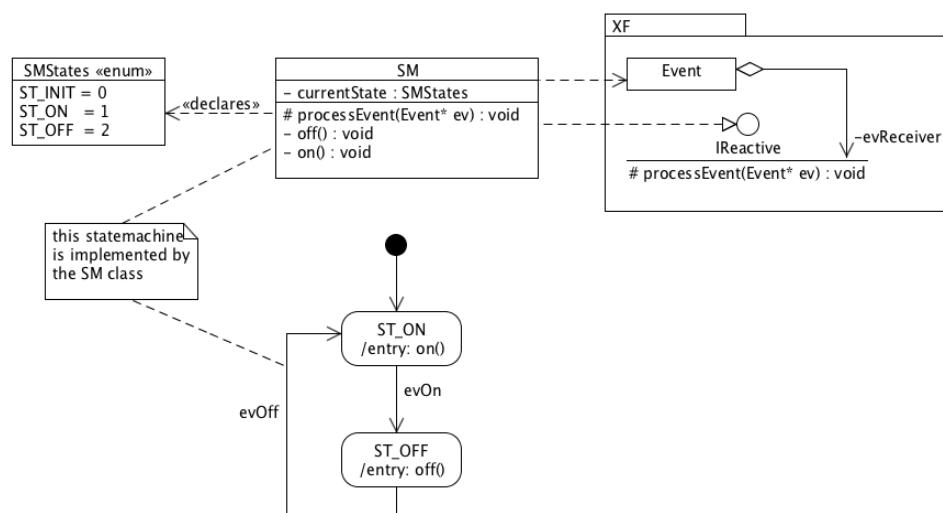


Figure 34 : Diagramme UML de la classe SM contenant une machine d'états transitions

```
#pragma once

#ifndef NULL
#define NULL 0
#endif

#include "XF/event.h"

typedef enum _SMStates
{
    ST_Init,
    ST_ON,
    ST_OFF
} SMStates;

typedef struct _SM
{
    SMStates currentState;
} SM;

void SM_init(SM * const me);
void SM_cleanup(SM * const me);
SM* SM_create();
void SM_destroy(SM* const me);

void SM_processEvent(SM* const me, Event* ev);
SMStates SM_getState(SM* const me);
void SM_on(SM* const me);
void SM_off(SM* const me);
|

#include "sm.h"

SM* SM_create()
{
    SM* newObj = (SM*) malloc(sizeof(SM));
    if (newObj != NULL)
    {
        SM_init(newObj);
    }
    return newObj;
}

void SM_destroy(SM* const me)
{
    if (me != NULL)
    {
        SM_cleanup(me);
    }
    free(me);
}

void SM_init(SM* const me)
{
    me->currentState = ST_Init;
}

void SM_cleanup(SM* const me)
{
}

void SM_processEvent(SM* const me, Event* ev) {...}
SMStates SM_getState(SM* const me)
{
    return me->currentState;
}

void SM_on(SM* const me)
{
    //switch something on
}

void SM_off(SM* const me)
{
    //switch something off
}
```

Figure 35 : Le code C de la classe SM, sans le code pour la méthode processEvent()

```
#include "sm.h"
#include "XF/event.h"
#include <stdio.h>

int main(void)
{
    SM* psm = SM_create();
    printf("SM, current state = %d\n", SM_getState(psm));
    Event* ev = Event_create();
    Event_setID(ev, evInit);
    SM_processEvent(psm, ev);
    printf("SM, current state = %d\n", SM_getState(psm));
    Event_setID(ev, evOff);
    SM_processEvent(psm, ev);
    printf("SM, current state = %d\n", SM_getState(psm));
    Event_setID(ev, evOn);
    SM_processEvent(psm, ev);
    printf("SM, current state = %d\n", SM_getState(psm));
}
```

Figure 36 : Le code C du programme de test pour la classe SM

Il est important de remarquer que l'Execution Framework (XF), qui constitue la couche d'abstraction pour l'utilisation des machines d'état n'est pas démarré ni utilisé. Cela n'entre pas dans le sujet de ce chapitre et sera détaillé plus tard. Les

événements sont envoyés directement à la méthode `processEvent()` afin de tester facilement tous les états. Le code suivant montre la méthode `processEvent()`:

```
void SM_processEvent(SM* const me, Event* ev)
{
    SMStates oldState;
    oldState = me->currentState;
    //this is the state calculator
    switch (me->currentState)
    {
        case ST_Init:
        {
            if (Event_getID(ev) == evInit)
            {
                me->currentState = ST_ON;
            }
        }
        case ST_ON:
        {
            if (Event_getID(ev) == evOff)
            {
                me->currentState = ST_OFF;
            }
        }
        case ST_OFF:
        {
            if (Event_getID(ev) == evOn)
            {
                me->currentState = ST_ON;
            }
        }
    }
    //this is the action calculator
    if (oldState != me->currentState)
    {
        switch (me->currentState)
        {
            case ST_ON:
            {
                if (oldState==ST_Init || oldState==ST_OFF)
                {
                    SM_on(me);
                }
            }
            case ST_OFF:
            {
                if (oldState==ST_ON)
                {
                    SM_off(me);
                }
            }
        }
    }
}
```

Figure 37 : Le code C de la méthode `processEvent()` de la classe `SM`

Comme l'on peut le voir, l'implémentation suit le pattern qui impose deux sections. Cette répartition facilite le débogage du code et donne plus de flexibilité dans la répartition des instructions dans les sections d'entrée et de sortie devant être exécutés durant les transitions.

Le pattern "Transition Method Pointer"

Ce pattern est montré sous forme orientée objets. Il pourrait par contre être implémenté sans autre sous forme C orientée objets. Aussi ce pattern part de l'idée que la classe avec la machine d'états transitions offre une interface `processEvent(Event* ev)`. Mais dans tout le reste, ce pattern se distingue fortement du pattern Double Switch montrée ci-dessus.

Le diagramme UML suivant montre la structure du pattern:

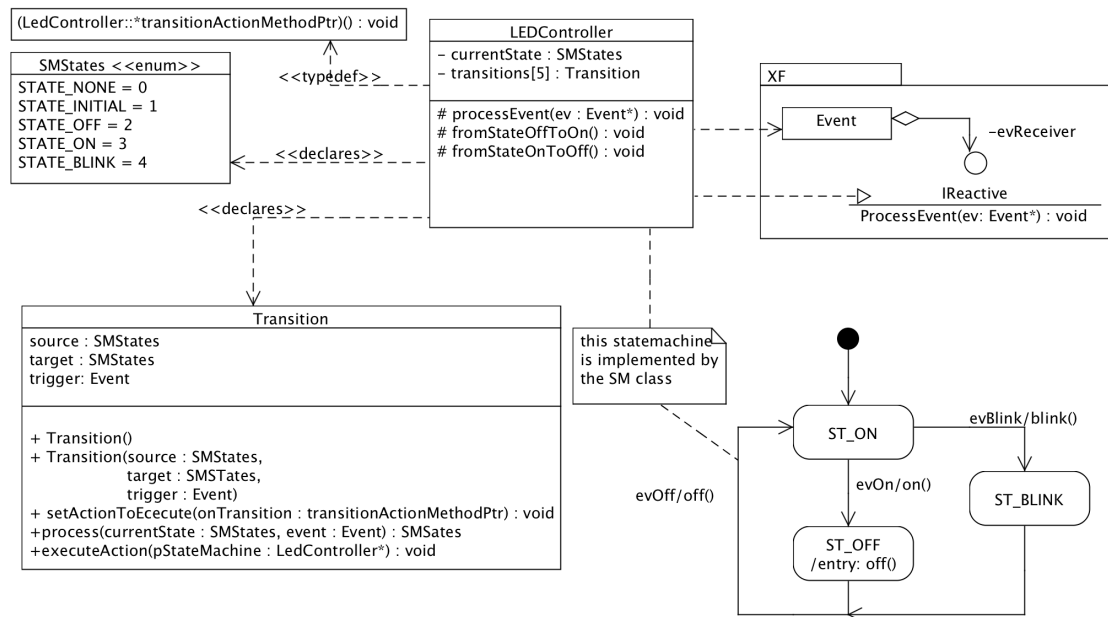


Figure 38 : La structure du pattern Transition Method Pointer

Dans le cas de ce pattern ne seront pas exécutés une Action on entry et une Action on exit d'un état, mais le code sera exécuté pendant la transition. Il s'agit donc d'un pattern d'Action on Transition. Dans la suite, le code des différentes classes du pattern sera montré.

```

#ifndef EVENT_H
#define EVENT_H

#include <QString>

class Event
{
public:
    Event(const QString & eventName = "none");

    bool operator==(const Event & event) const;
    const Event & operator=(const Event & event);

protected:
    QString _eventName;
};

#endif // EVENT_H

#include "event.h"

Event::Event(const QString & eventName)
    : _eventName(eventName)
{
}

bool Event::operator==(const Event & event) const
{
    return (_eventName == event._eventName);
}

const Event & Event::operator=(const Event & event)
{
    _eventName = event._eventName;

    return *this;
}

```

Figure 39 : La déclaration et la définition de la classe Event

Comme l'on peut voir, un évènement sera identifié par son nom. Dans l'exemple ici, l'attribut est implémenté par la classe `QString`.

```
#ifndef LEDCONTROLLER_H
#define LEDCONTROLLER_H

#include "event.h"

class LedController
{
public:
    LedController();

    void initialize();
    void startStateMachine();

    void processEvent(const Event & event);

protected:
    typedef enum { ... } SMStates;

    // Define method pointer type
    typedef void (LedController::*transitionActionMethodPtr)();

    class Transition { ... };

    // Transition methods
protected:
    void fromStateOffToOn();
    void fromStateOnToOff();

protected:
    void traceCurrentState();

protected:
    Transition transitions[5];
    SMStates _currentState;
};

#endif // LEDCONTROLLER_H
```

Figure 40 : La déclaration de la classe `LEDController`

Comme l'on peut voir, à l'intérieur de la classe sont définis les états, la classe `Transition` ainsi qu'un prototype de pointer de méthode. Remarquons la déclaration de ce prototype de pointeur :

```
// Define method pointer type
typedef void (LedController::*transitionActionMethodPtr)();
```

Figure 41 : La déclaration du pointeur de méthode

Ce dernier sera plus tard utilisé pour faire appel à des méthodes lesquelles exécutent le code d'action durant les transitions. Ces méthodes sont `fromStateOffToOn()` et `fromStateOnToOff()`. Les pointeurs vers ces deux méthodes seront enregistrés dans les transitions correspondantes. Ceci sera montré dans l'explication de la méthode `LEDController::initialize`.

```
typedef enum
{
    STATE_NONE,
    STATE_INITIAL,
    STATE_OFF,
    STATE_ON,
    STATE_BLINK
} SMStates;
```

Figure 42 : La déclaration des états

Les états de la machine d'états transitions sont déclarés comme d'habitude dans un enum.

La classe `Transition` est déclarée comme suit :

```
class Transition
{
public:
    Transition()
    {
        this->source = STATE_NONE;
        this->target = STATE_NONE;
        onTransition = nullptr;
    }

    Transition(const SMStates & source, const SMStates & target, const Event & event)
    {
        this->source = source;
        this->target = target;
        trigger = event;
        onTransition = nullptr;
    }

    void setActionToExecute(transitionActionMethodPtr onTransition)
    {
        this->onTransition = onTransition;
    }

    SMStates process(const SMStates & currentState, const Event & event)
    {
        if (currentState == source && // Check if current state matches
            event == trigger) // Check if trigger matches
        {
            // Change to next state
            return target;
        }

        return STATE_NONE;
    }

    void executeAction(LedController * pStateMachine)
    {
        if (onTransition)
        {
            // Call method of state machine using method pointer
            (pStateMachine->*onTransition)();
        }
    }

public:
    SMStates source; // From
    SMStates target; // To
    Event trigger; // Event that causes the state change

    transitionActionMethodPtr onTransition;
};
```

Figure 43 : La déclaration de la classe `Transition`

Concernant la classe `Transition` l'on peut faire la remarque celle connaît pour chaque transition l'état de départ, l'état d'arrivé ainsi que l'événement qui

déclenche la transition. En plus, elle peut enregistrer l'adresse d'une méthode qui exécute le code de l'Action on Transition.

```
LedController::LedController()
{
}

void LedController::initialize()
{
    // Define state transitions
    transitions[0] = Transition(STATE_INITIAL, STATE_OFF, Event("initial"));
    transitions[1] = Transition(STATE_OFF, STATE_ON, Event("evOn"));
    transitions[2] = Transition(STATE_ON, STATE_OFF, Event("evOff"));
    transitions[3] = Transition(STATE_OFF, STATE_BLINK, Event("evBlink"));
    transitions[4] = Transition(STATE_BLINK, STATE_OFF, Event("none"));

    // Define which code to execute on transitions
    transitions[1].setActionToExecute(&LedController::fromStateOffToOn);
    transitions[2].setActionToExecute(&LedController::fromStateOnToOff);

    _currentState = STATE_INITIAL;

    startStateMachine();
}

void LedController::startStateMachine()
{
    traceCurrentState();           // Trace current state
    processEvent(Event("initial")); // TODO: Event should be pushed to the XF instead!
}

void LedController::processEvent(const Event & event)
{
    SMStates nextState = STATE_NONE;

    // Iterate over available transitions
    for (uint32_t i = 0; i < sizeof(transitions); i++)
    {
        // Let the transition process the event. In case a transition takes place,
        // the transition will return the new/next state.
        nextState = transitions[i].process(_currentState, event);

        if (nextState != STATE_NONE)
        {
            transitions[i].executeAction(this); // Execute action on transition
            _currentState = nextState;          // Change to new state
            traceCurrentState();                // Trace current state
            break;                             // We are done for actual event
        }
    }
}
```

Figure 44 : Le code important de la classe LEDController

L'on peut voir comment la méthode `LEDController::initialize` définit toute la machine d'états transitions. Le détail de la machine d'états transitions définie de cette façon est montre dans la figure ci-dessous :

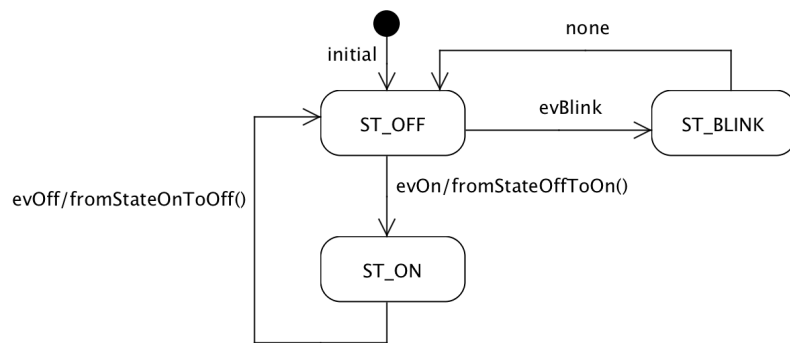


Figure 45 : La machine d'états transitions définie par la classe LEDController

La prochaine figure montre une façon alternative de représenter cette même machine d'états transitions :

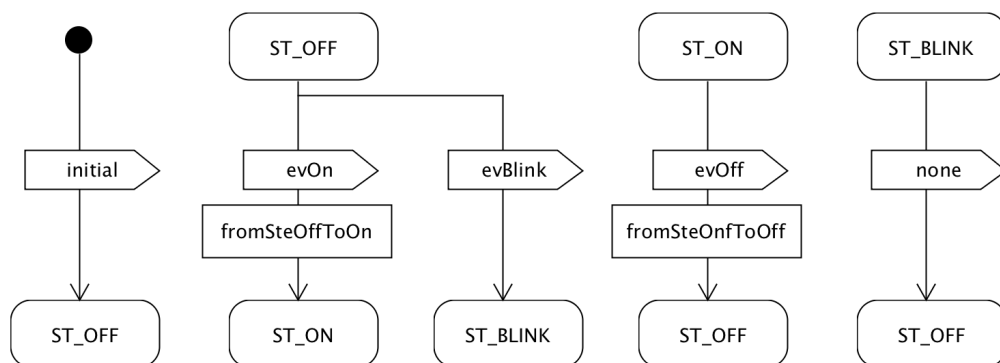


Figure 46 : Représentation de la machine d'états transitions de la classe LEDController en notation SDL

Cette représentation est d'ailleurs parfaitement utile quand on veut utiliser le Transition Method pattern.

La méthode `LEDController::processEvent` distribue les évènements à la machine d'états transitions et en conséquence, le code de transition correspondant sera exécuté. L'appel de la méthode elle-même est effectué par un objet de la classe `Transition`, parce que c'est cette dernière qui connaît les adresses des méthodes à exécuter. La méthode `Transition::executeAction()` est responsable pour ceci. Remarquez bien que les méthodes appelées doivent être des méthodes de la class `LEDController`. Ceci est parce que le prototype du pointeur de méthode est déclaré de cette façon. Ceci fait bien du sens, parce qu'il s'agit bien de la machine d'états transition définissant le comportement de la classe `LEDController`.

Pour finir, il ne faut pas oublier de dire que ce pattern peut sans autre être implémenté en C orienté objets.

Des pattern de découplage

Dans ce chapitre seront décrits des pattern qui favorisent le découplage des éléments. Les pattern suivants peuvent être mentionnés :

- Subject – Observer
- Model View Controller
- Adapter

Dans le cadre de ce manuscrit seront discutés seulement deux variantes du pattern Subject – Observer. Ce pattern peut être décrit comme suit :

Problem	Guarantee synchronization between the state (data) of an object (subject) and its observers (representation, reaction on changes).
Solution	A class Subject (method subscribe) and a class Observer (method notify). An observer may subscribe to a subject. a subject may then call the notify method of the observer. This is some kind of enhanced call back system. Specific subjects and specific observers are created by deriving them from Subject and Observer. Polymorphism is used to isolate specific observers from specific subjects.
Consequences	Very high degree of modularity. Packages communication by the mean of the observer pattern are highly independent from each other.

Tableau 2 : Description du pattern observateur

Subject – Observer orienté objets

Dans cette section sera présenté la variante orienté objets du pattern Subject – Observer. La figure suivante montre la structure du pattern en diagramme de classe UML.

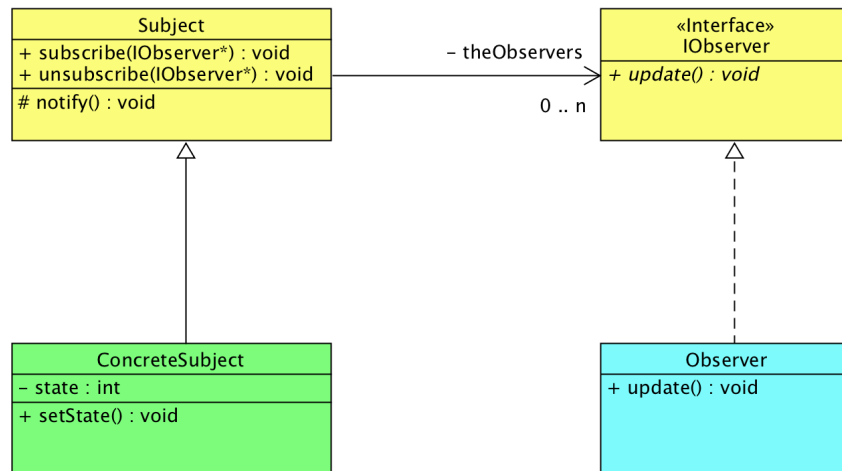


Figure 47 : Pattern observateur orienté objet représenté en UML

Son comportement est montré par le diagramme de séquence suivant :

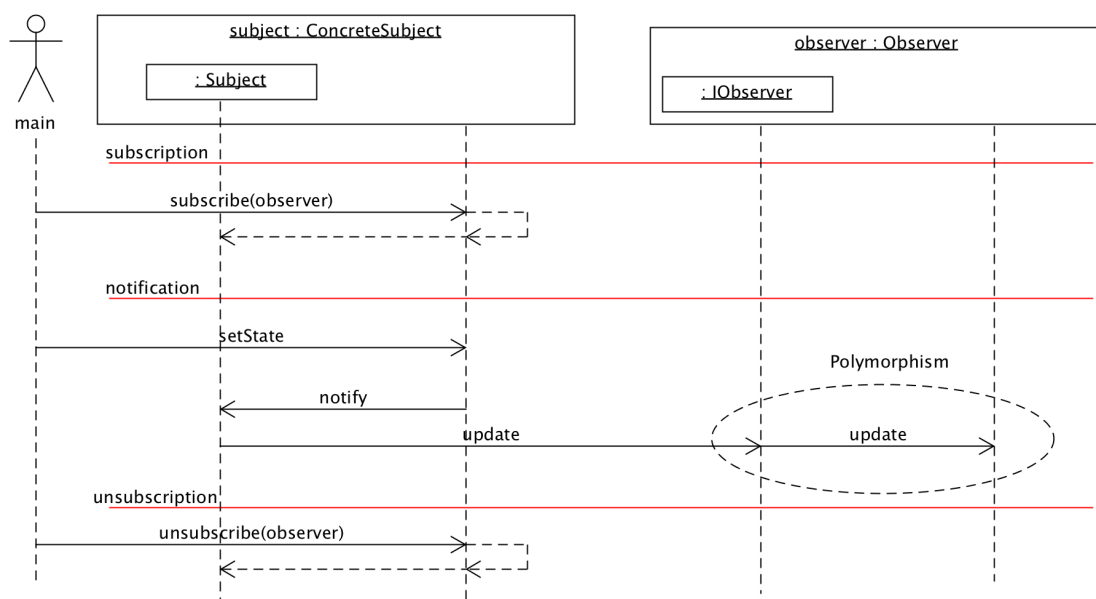


Figure 48 : Comportement du pattern Subject Observer

Il est bien visible que ce pattern fait appel à deux principes orientés objets : l'héritage et le polymorphisme. D'une part, la classe **Subject** offre les méthodes d'inscription et de désinscription d'observateurs dans ou de la liste des observateurs. Elle offre aussi le mécanisme de notification. En cas d'un changement dans le sujet, les observateurs inscrits seront notifiés. La méthode `IObservable::update()` est polymorphe et doit être redéfinie par la classe **Observer**. Le code d'un exemple de ce pattern est montré dans les figures suivantes.

```
#ifndef SUBJECT_H
#define SUBJECT_H

#define MAX 10
#include "observer.h"

class Subject
{
public:
    Subject();
    void subscribe(IObserver* obs);
    void unsubscribe(IObserver* obs);
protected:
    void notify();
private:
    IObserver* obsList[MAX];
};

#endif // SUBJECT_H

#include "subject.h"
#include <iostream>

Subject::Subject()
{
    for (int i=0; i<MAX; i++)
        obsList[i]=NULL;
}

void Subject::subscribe(IObserver *obs)
{
    for (int i=0; i<MAX; i++)
        if (obsList[i]==NULL)
        {
            obsList[i]=obs;
            break;
        }
}

void Subject::unsubscribe(IObserver *obs)
{
    for (int i=0; i<MAX; i++)
        if (obsList[i]==obs)
        {
            obsList[i]=NULL;
        }
}

void Subject::notify()
{
    for (int i=0; i<MAX; i++)
        if (obsList[i]!=NULL)
            obsList[i]->update();
}
```

Figure 49 : Code C++ de la classe Subject

```
#ifndef BUTTON_H
#define BUTTON_H

#include "subject.h"

class Button : public Subject
{
public:
    Button();
    void setState(int state);
private:
    int state;
};

#endif // BUTTON_H

#include "button.h"

Button::Button()
{
    state = 0;
}

void Button::setState(int state)
{
    int oldState = this->state;
    this->state = state;
    if (oldState != this->state)
        this->notify();
}
```

Figure 50 : Code C++ de la classe Button

```
#ifndef OBSERVER_H
#define OBSERVER_H

class IObserver
{
public:
    virtual void update() = 0;
};

#endif // OBSERVER_H

#ifndef BUTTONMANAGER_H
#define BUTTONMANAGER_H

#include "observer.h"

class ButtonManager : public IObserver
{
public:
    ButtonManager();
    void update();
};

#endif // BUTTONMANAGER_H

#include "buttonmanager.h"
#include <iostream>

ButtonManager::ButtonManager()
{
    //initialization code goes here
}

void ButtonManager::update()
{
    std::cout <<"State of Button has changed\n";
}
```

Figure 51 : Code C++ de l'interface IObserver et de la classe ButtonManager

```
#include "button.h"
#include "buttonmanager.h"
#include <iostream>

int main(int argc, char *argv[])
{
    Button but;
    ButtonManager bMan;

    but.subscribe(&bMan);
    but.setState(1);
    but.setState(0);
    but.setState(0); //will not notify
    std::cout <<"one more observer\n";
    ButtonManager otherBMan;
    but.subscribe(&otherBMan);
    but.setState(1);
    std::cout <<"unsubscribe one observer\n";
    but.unsubscribe(&bMan);
    but.setState(0);
}
```

Figure 52 : Code C++ du programme de test pour le gestionnaire de boutons

L'utilisation du polymorphisme est un des désavantages de ce pattern et implique qu'il peut seulement être implémenté en utilisant un langage orienté objets. Pour aussi donner suite à des langages non orientés objets il y existe une deuxième variante non orienté objets. Cette dernière sera décrite dans la section suivante.

Subject - Observer non orienté objet

Dans cette section sera présenté la variante non orienté objet du pattern Subject – Observer.

La figure suivante montre la structure du pattern en diagramme de classe UML.

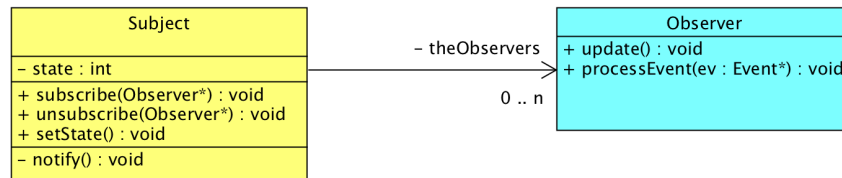


Figure 53 : Pattern observateur non orienté objet représenté en UML

Le polymorphisme et l'héritage ne seront donc pas utilisés par ce pattern. Dans des systèmes embarqués, il est souvent important que les différents éléments du système soient accessibles à tout moment. Cela implique que les messages soient échangés de façon asynchrone. La figure suivante montre le comportement asynchrone de ce pattern :

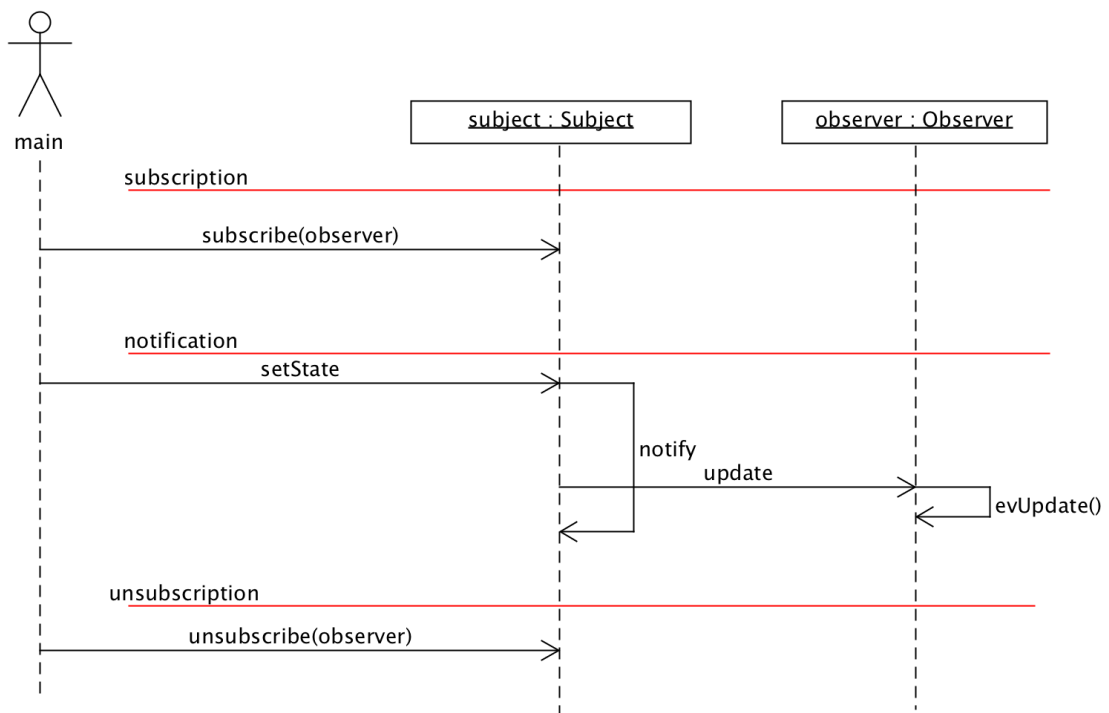


Figure 54 : Comportement asynchrone du pattern Subject – Observer non orienté objets

Dans cette variante, le découplage est effectué par l'envoi d'un évènement suite à l'appel de la méthode `Observer::update()` par le sujet. De cette façon, le sujet est immédiatement à nouveau disponible. L'observateur s'auto envoie un évènement `evUpdate()` et il peut alors traiter le message dès qu'il en a le temps. Le découplage du sujet et de l'observateur est dans ce cas la effectué par le moteur évènementiel comme par exemple le XF.

Il est clair que dans cette variante du pattern l'observateur doit disposer d'une machine d'états transition pour pouvoir réagir à l'événement qui lui est envoyée par le sujet. La figure suivante montre cette dernière.

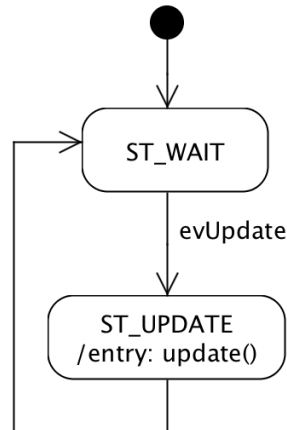


Figure 55 : Machine d'états transitions pour le pattern Subject – Observer non orienté objets

Exercice pratique

La classe `Button` dispose d'une méthode `toggleState()`. Par l'appel de cette méthode, la valeur d'un attribut est commuté de 0 à 1 respectivement de 1 à 0. Tout changement de la valeur de l'attribut est signalé aux trois observateurs du type `ButtonHandler`. Ceci s'effectue à l'aide de la méthode `ButtonHandler::onButtonChanged()`.

Tâches :

- Développez un diagramme de séquence `SD_BChange` qui reflète la situation décrite ci-dessus.
- Dérivez de `SD_BChange` un diagramme de classe. Utilisez le pattern Subject – Observer orienté objets. Développez un nouveau diagramme de séquence `SD_BChangeOO`
- Dérivez de `SD_BChange` un diagramme de classe. Utilisez le pattern Subject – Observer non orienté objets. Développez un nouveau diagramme de séquence `SD_BChangeNONOO`
- Développez un diagramme d'objets pour le cas non orienté objets
- Implémentez le cas non orienté objets en utilisant QT.

A part du pattern Subject-Observer ils y existent encore d'autres pattern pour le découplage étant plus ou moins génériques. Un mécanisme de découplage très connu est la méthode du callback. Cette dernière peut être implémentée de façons différentes et comme toujours il y a une variante orientée-objets et une variante non orientée objets.

Le pattern callback

Le tableau ci-dessous décrit le pattern callback :

Problem	Enable an object (Caller) to notify a single other object (Called) about an event. The event can be anything of interest that the Caller must tell the Called.
Solution	The caller object contains a pointer to a method or function of the called object. It also contains a pointer to the object itself. The method or function pointer is initialized with the address of the callback function / method. This is called in order to achieve notification.
Consequences	Quite good degree of modularity. The Caller and the Called are coupled by the class name but not by the method name. Enables synchronous communication between packages without coupling them too much. Only works well in a one to one relation between Caller and Called. But may also work with one to many configurations.

Tableau 3 : Description du pattern callback

Une variante orientée-objets du pattern callback

Dans la suite, il sera montré une variante orientée-objets du pattern callback. Il est un peu difficile d'exprimer ce pattern en diagramme UML de classe, car il ne possède pas d'éléments orientés-objets exprimés. Ceci se sent bien dans la figure ci-dessous. Cette difficulté se base dans le fait que la relation "un-à-un" entre la classe `Caller` et `Called` ne nécessite pas du tout d'éléments orientés-objets. Mais il est quand-même possible de réduire le couplage entre `Caller` et `Called` au moyen d'un héritage de façon non négligeable. La classe `Caller` ne doit pas du tout connaître le nom de la classe `Called`, il suffit si elle connaît la classe `CallbackEnabler`. La classe `Called` (ne) hérite (rien) de cette classe. Mais ceci a comme effet que la classe `Called` devient un `CallbackEnabler`, car l'héritage est une relation "est-un".

De plus, la classe `CallbackEnabler` déclare le prototype du pointeur vers la méthode callback et la classe `CBElement`. Ceci a de nouveau comme effet qu'un léger couplage entre les deux classes. Mais ce couplage est acceptable car il se restreint au nom de la classe. La figure ci-dessous montre le diagramme de classe en UML :

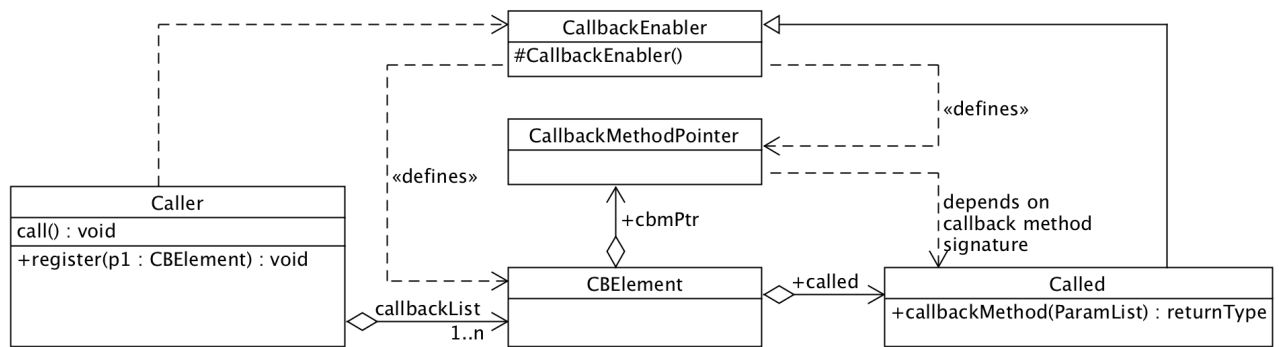


Figure 56 : Diagramme UML du pattern callback

Les figures suivantes montrent une façon possible d'implémenter ce pattern en langage C++ :

```
#ifndef CALLED_H
#define CALLED_H

#include "cbenabler.h"

class Called : public CBEabler
{
public:
    Called();
    // CallbackInterface interface
    void callbackMethod();
};

#endif // CALLED_H

#include "called.h"
#include <iostream>

using namespace std;

Called::Called()
{
}

void Called::callbackMethod()
{
    cout <<this << " has been called" <<endl;
}

#ifndef CALLER_H
#define CALLER_H

#include "cbenabler.h"

class Caller
{
public:
    //methods
    Caller();
    void registerCB(const CBEabler::CBElement& p1);
    void call();
private:
    CBEabler::CBElement callback;
};

#endif // CALLER_H

#include "caller.h"
#include "called.h"

Caller::Caller()
{
}

void Caller::registerCB(const CBEabler::CBElement& p1)
{
    callback=p1;
}

void Caller::call()
{
    (callback.called->*callback.cbmPtr)();
}
```

Figure 57 : Le code du pattern callback en C++ orienté-objets montrant les classes Called et Caller

```
#ifndef CBENABLER_H
#define CBENABLER_H

//minimal coupling to the
//caller class
class Called;

class CBenabler
{
public:
    //definitions
    typedef void (CBenabler::*cbMethodPtr)();

    typedef struct CBElement
    {
        Called* called;
        cbMethodPtr cbmPtr;
    } CBElement;

    //inhibit public instantiation
protected:
    CBenabler() {};
};
#endif // CBENABLER_H

#include "called.h"
#include "caller.h"
#include "cbenabler.h"

int main()
{
    Caller theCaller;
    Called theCalled;
    CBenabler::CBElement cbElement;
    cbElement.called=&theCalled;
    cbElement.cbmPtr=(CBenabler::cbMethodPtr)&Called::callbackMethod;
    theCaller.subscribe(cbElement);
    theCaller.call();
}
```

Figure 58 : Le code du pattern callback en C++ orienté-objets montrant la classe CBenabler et le programme de test

Ce pattern a comme propriété qu'il peut très facilement être implémenté en C, donc en langage non orienté-objets. Une implémentation possible de ce pattern en "C orienté-objets" est montrée par les figures suivantes.

```
#ifndef CALLED_H
#define CALLED_H

typedef struct Called
{
} Called;

Called* Called_create();
void Called_destroy(Called* me);
void Called_init(Called* me);
void Called_cleanup(Called* me);
void Called_callbackFunction(Called* me);

#endif // CALLED_H
```

```
#ifndef CALLER_H
#define CALLER_H

#include "cbenabler.h"

typedef struct Caller
{
    //this is the callbackinformation
    CBElement callback;
} Caller;

Caller* Caller_create();
void Caller_destroy(Caller* me);
void Caller_init(Caller* me);
void Caller_cleanup(Caller* me);

void Caller_registerCB(Caller* me, const CBElement* p1);
void Caller_call(Caller* me);

#endif // CALLER_H
```

```
#include "called.h"
#include <stdio.h>
#include <stdlib.h>

Called *Called_create()
{
    Called* retval;
    retval=(Called*) malloc(sizeof(Called));
    if (retval != NULL)
    {
        Called_init(retval);
    }
    return retval;
}

void Called_destroy(Called *me)
{
    Called_cleanup(me);
    free(me);
}

void Called_init(Called *me){}
void Called_cleanup(Called *me){}

void Called_callbackFunction(Called *me)
{
    printf("%p has been called\n", (void*) me);
}

#include "caller.h"
#include <stdlib.h>

Caller *Caller_create()
{
    Caller* retval;
    retval=(Caller*) malloc(sizeof(Caller));
    if (retval != NULL)
    {
        Caller_init(retval);
    }
    return retval;
}

void Caller_destroy(Caller *me)
{
    Caller_cleanup(me);
    free(me);
}

void Caller_init(Caller *me){}
void Caller_cleanup(Caller *me){}

void Caller_registerCB(Caller* me, const CBElement* p1)
{
    me->callback = *p1;
}

void Caller_call(Caller *me)
{
    (*me->callback.cbFPtr)(me->callback.called);
}
```

**Figure 59 : Le code du pattern callback en "C orienté-objets"
montrant les classes Called et Caller**

```
#ifndef CBENABLER_H
#define CBENABLER_H

struct Called;
//this is the callback function prototype
typedef void (*cbFunctionPtr)();

typedef struct CBElement
{
    //this is the callbackfunction
    cbFunctionPtr cbfPtr;
    //this is the called object
    struct Called* called;
} CBElement;

#endif // CBENABLER_H

#include "caller.h"
#include "called.h"
#include "cbenabler.h"

int main()
{
    Caller* caller = Caller_create();
    Called* called = Called_create();

    CBElement cbe;
    cbe.called = called;
    cbe.cbfPtr = &Called_callbackFunction;
    Caller_registerCB(caller,&cbe);
    Caller_call(caller);

    Called_destroy(called);
    Caller_destroy(caller);

    return 0;
}
```

Figure 60 : Le code du pattern callback en "C orienté-objets" montrant la classe CBenabler et le programme de test

Il est remarquable combien peu les classes `Caller` et `Called` sont couplés dans les deux implémentations. Ceci est un avantage indéniable de ce pattern. De plus, il n'est pas polymorphe et tous les liens sont calculés au temps de compilation. Ceci résulte dans du code machine très rapide et compact.

Il doit aussi être remarqué à cet endroit qu'ils y existent des variantes supplémentaires de ce pattern comme par exemple le Callback Method Pattern et le Callback Interface Pattern. Ces derniers ne seront par contre pas traités dans le cadre de ce cours.

Des pattern pour l'instanciation d'objets systèmes

Dans cette section seront montrés des pattern qui rendent possible la création d'objets uniques au niveau du système.

Le pattern Singleton

Le pattern singleton peut être décrit comme suit :

Problem	Guarantee that in any circumstances only one instance of a given class is created.
Solution	A static attribute of type <pointer to the class herself> and a static method getInstance() that return a pointer to the one and only object of this class. This method tests, if the attribute contains an valid address or not. If not, she cerates an object of the class and stores the address of the object into the attribute and returns the address, if yes she simply returns the address.
Consequences	Compact memory usage and no "accidently" object creation.

Tableau 4 : Description du pattern Singleton

Le pattern Singleton orienté-objets

Le pattern Singleton peut être implémenté en orienté-objets de très belle façon : La représentation de cette variante en UML est montrée ci-dessous :

Singleton
+ static getInstance() : Singleton&
- Singleton()
- Singleton(other : const Singleton&)
- Singleton& operator=(other : const Singleton&)

Figure 61 : Le pattern Singleton en UML

```
#include <iostream>
using namespace std;

class Singleton
{
public:
    static Singleton& getInstance()
    {
        static Singleton instance;
        return instance;
    }
    void doSomething()
    {
        cout <<"Hello, I am " <<this <<endl;
    }

private:
    //this will prevent creation of instance
    Singleton() {}
    Singleton(const Singleton&) {}
    void operator=(const Singleton&) {}
};

int main()
{
    Singleton::getInstance().doSomething();
    Singleton::getInstance().doSomething();
    Singleton::getInstance().doSomething();

    // Singleton s;    //this will not work
    return 0;
}
```

Figure 62 : Le code du pattern Singleton orienté objets

Comme l'on peut voir, une variable statique est créée à l'intérieur de la méthode `Singleton::getInstance()` et il est rendu une référence de cette variable comme résultat de l'appel de la fonction. Ce n'est donc pas un attribut de la classe qui enregistre l'adresse de l'objet.

Le pattern Singleton non orienté-objets

Le pattern Singleton peut aussi être implémenté en non orienté objets. La représentation en UML est montrée ci-dessous :

Singleton
- instance : Singleton*
+ getInstance() : Singleton*
- Singleton() - Singleton(other : const Singleton&) - Singleton& operator=(other : const Singleton&)

Figure 63 : Le modèle du pattern Singleton non orienté objets

Cette variante permette aussi une implémentation en C++. Le code en C est montré de suite :

```
#include <stdio.h>
#include <assert.h>

typedef struct Singleton
{
    //other attributs;
} Singleton;

static Singleton* theOnlyInstance = NULL;

void Singleton_init(Singleton* theObj)
{
    //verify init is called only once
    assert(theOnlyInstance==NULL);
    theOnlyInstance = theObj;
}

//this will be a private function
Singleton* Singleton_getInstance()
{
    //verify that init was called
    assert(theOnlyInstance!=NULL);
    return theOnlyInstance;
}

void Singleton_display()
{
    //all methods must use
    //Singleton_getInstance to access data
    printf("Object address %p\n", Singleton_getInstance());
}

int main()
{
    Singleton mySingleton;
    Singleton_init(&mySingleton);

    //this will not work
    //Singleton_init(&mySingleton);

    Singleton_display();
    Singleton_display();

    return 0;
}
```

Figure 64 : Le code du pattern Singleton non orienté objets

Remarquez que cette variante n'utilise pas des variables statiques. Le développeur doit créer lui-même une variable globale pour l'instance. L'adresse de cette dernière sera ensuite enregistrée par la fonction `Singleton_init()`. Toutes les fonctions de la classe doivent utiliser la fonction `Singleton_getInstance()` pour accéder aux données de l'objet.

Pattern pour la création et l'initialisation de systèmes

Ces pattern font partie des pattern de création. Dans le cadre de ce cours, le pattern Singleton avait déjà été présenté. Il sert à la création d'objets uniques. Le pattern qui sera présenté en suite sert à la création de systèmes entiers. Ceci couvre la création des objets du système et leur initialisation ainsi que le lien des objets entre eux.

Le mécanisme de la création et de l'initialisation est appelé "factoring" et le mécanisme du lien est appelé "building".

Le pattern Factory

Deux pattern qui sont en mesure d'effectuer exactement ce qui vient d'être dit sont le pattern "Abstract Factory" et le pattern "Builder". Pour rendre l'affaire plus simple et compact pour des systèmes embarqués il sera présenté ici un pattern Factory qui s'occupe en même temps du "factoring" et du "building".

Le tableau suivant décrit le pattern :

Problem	Create all objects of a system und link them together in order to enable communication between them.
Solution	A class creates all objects of the system (factoring). The class assumes that all of them dispose on an initialize method in order to initialize data. The class also links all the objects (building). This assumes that all objects dispose on an initializeRelations method. The class generally also gives access to the objects by the means of a getter method. It may provide a create and a build method or one combined method.
Consequences	This pattern assumes the methods initialize and initializeRelations without prescribing them in an interface. Therefore, it is a pattern based on conventions that must be respected.

Tableau 5 : Description du pattern Factory

Le diagramme de classes en UML pour ce pattern est montré dans la figure ci-dessous :

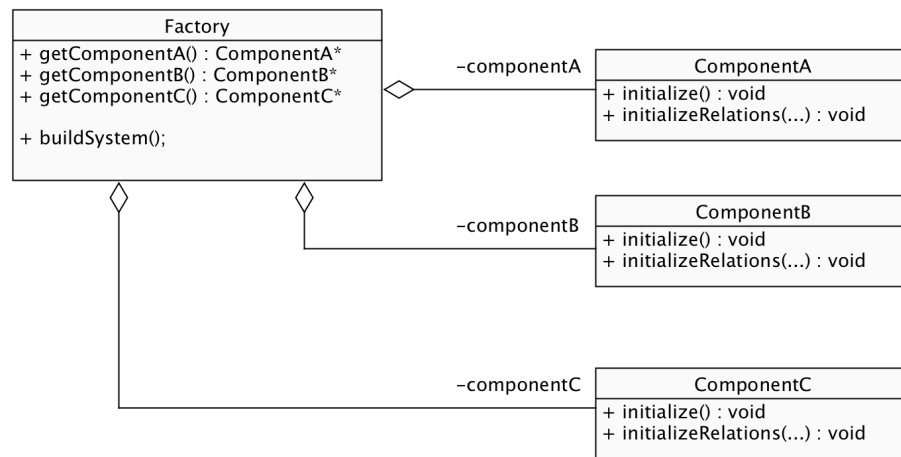


Figure 65 : Diagramme UML de classes montrant le pattern Factory

Les méthodes `getComponent` rendent possible l'accès aux objets du système. Elles ont d'ailleurs un effet comme le pattern Singleton, si l'on le désire. La méthode `buildSystem` est une méthode combinée de "factoring" et de "building". Il est tout de suite visible que ce pattern ne contient aucun élément orienté-objets et qu'il peut donc être implémenté soit en C++ soit en C ou si l'on veut en langage orienté-objets ou bien en langage non orienté-objets.

Le diagramme de séquence ci-dessous montre une application typique de ce pattern :

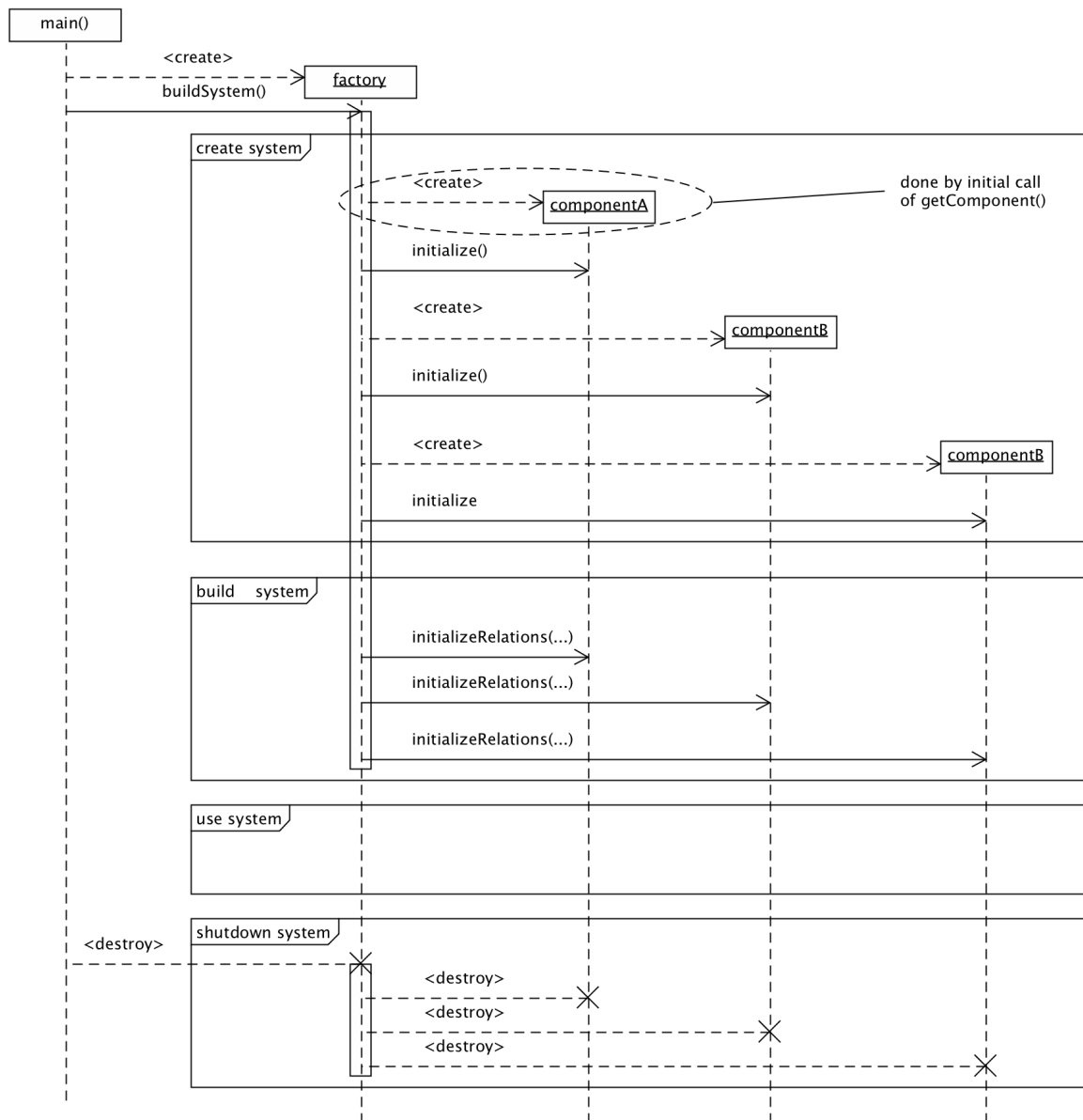


Figure 66 : Diagramme UML de séquence du pattern Factory

Remarquez que ce pattern est parfaitement utile pour construire des scénarios de test. Tout est faisable entre des tests d'unité par des tests de sous-systèmes jusqu'à des test de système. Il doit juste être mise en place la classe Factory correspondante.

Remarquez aussi que la méthode prescrite `initializeRelations` est utilisée pour réaliser la technique de la "Dependency Injection". Cette dernière consiste dans le fait que les liens entre les objets sont créés de façon explicite par le développeur et ne pas de façon automatique ou il est assumé que les objets doit trouver leurs liens eux-mêmes et qui corresponde à l'opinion commun. Des systèmes créés par la "Dependency Injection" sont en règle générale plus stables et plus gérables. De plus il est aussi réalisé une espèce de Save Pointer Pattern. Ce pattern aide à éviter l'utilisation des pointeurs invalides.

Le code suivant démontre un exemple du pattern Factory implementé en C++ :

```
#include <iostream>
using namespace std;

class Display
{
private:
    //...
public:
    Display()
    {
        cout << "Display created" << endl;
    }
    void initialize() {}
};

class Computer
{
private:
    Display* theDisplay;
public:
    Computer()
    {
        theDisplay = NULL;
        cout << "Computer created" << endl;
    }
    void initialize() {}
    void initializeRelations(Display* p)
    {
        theDisplay = p;
    }
};

class Keyboard
{
private:
    Computer* theComputer;
public:
    Keyboard()
    {
        theComputer = NULL;
        cout << "Keyboard created" << endl;
    }
    void initialize() {}
    void initializeRelations(Computer* p)
    {
        theComputer = p;
    }
};

class Factory
{
private:
    Keyboard* pKeyboard;
    Computer* pComputer;
    Display* pDisplay;
public:
    Factory()
    {
        pKeyboard = NULL;
        pComputer = NULL;
        pDisplay = NULL;
    }
    ~Factory()
    {
        delete pKeyboard;
        delete pComputer;
        delete pDisplay;
    }
    Keyboard* getKeyboard()
    {
        if (!pKeyboard) pKeyboard = new Keyboard();
        return pKeyboard;
    }
    Computer* getComputer()
    {
        if (!pComputer) pComputer = new Computer();
        return pComputer;
    }
    Display* getDisplay()
    {
        if (!pDisplay) pDisplay = new Display();
        return pDisplay;
    }

    void buildSystem()
    {
        cout << "Initializing system" << endl;
        getKeyboard()->initialize();
        getComputer()->initialize();
        getDisplay()->initialize();
        cout << "Setting relations" << endl;
        getKeyboard()->initializeRelations(pComputer);
        getComputer()->initializeRelations(pDisplay);
        cout << "System is ready" << endl;
    }
};

int main()
{
    Factory f;
    f.buildSystem();
    //go on
    return 0;
}
```

Figure 67 : Le code C++ d'un exemple du pattern Factory

Remarquez bien comment le constructeur et les méthodes `get` sont utilisées pour implémenter de façon élégante le pattern Singleton pour les classes `Keyboard`, `Computer` et `Display`.

A ce point-là le chapitre sur les pattern embarqués est terminé. Dans le chapitre suivant traitera les chaînes d'outils pour le développement des systèmes embarqués.